

[Week08] <자료구조 구현 - 연결리스트와 스택>

Challenge1. 단순 연결리스트 구현

```
#include <stdio.h>
#include <stdlib.h>

// 노드 구조체 정의
typedef struct Node {
    int data;
    struct Node* next;
} Node;

// 전역 변수로 리스트의 시작점을 NULL로 초기화
Node* head = NULL;

// 노드 추가 함수
void append(int data) {

    Node* newNode = (Node*)malloc(sizeof(Node));
    if (newNode == NULL) {
        printf("메모리 할당 실패!\n");
        return;
    }
    newNode->data = data;
    newNode->next = NULL;

    if (head == NULL) {
        head = newNode;
        return;
    }

    Node* temp = head;
    while (temp->next != NULL) {
        temp = temp->next;
    }
}
```

```

    }
    temp->next = newNode;
}

// 노드 삭제 함수
void delete_node(int key) {
    Node* temp = head;
    Node* prev = NULL;

    if (temp != NULL && temp->data == key) {
        head = temp->next;
        free(temp);
        return;
    }

    while (temp != NULL && temp->data != key) {
        prev = temp;
        temp = temp->next;
    }

    if (temp == NULL) return;

    prev->next = temp->next;
    free(temp);
}

// 전체 리스트 출력
void print_list() {
    Node* temp = head;
    printf("\n[현재 연결 리스트 상태]\n");
    while (temp != NULL) {
        printf("[주소: %p | 데이터: %d | Next: %p] -> \n",
            (void*)temp, temp->data, (void*)temp->next);
        temp = temp->next;
    }
    printf("NULL\n\n");
}

```

```

}

// 전체 메모리 해제
void free_all() {
    Node* temp = head;
    while (temp != NULL) {
        Node* next_node = temp->next;
        free(temp);
        temp = next_node;
    }
    head = NULL;
    printf("모든 노드 메모리 해제 완료.\n");
}

int main() {
    printf("--- 1. 노드 5개 추가 ---\n");
    append(10); append(20); append(30); append(40); append
(50);
    print_list();

    printf("--- 2. 노드 '30' 삭제 ---\n");
    delete_node(30);
    print_list();

    free_all();

    return 0;
}

```

Challenge 2. 스택 (Stack - LIFO) 구현

```

#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node* next;
} Node;

Node* top = NULL;

void push(int data) {
    Node* newNode = (Node*)malloc(sizeof(Node));
    if (newNode == NULL) return;

    newNode->data = data;
    newNode->next = top;
    top = newNode;
    printf("Push: %d\n", data);
}

int pop() {
    if (top == NULL) {
        printf("오류: 스택이 비어 있습니다! (Stack Underflow)\n");
        return -1;
    }

    Node* temp = top;
    int pop_data = temp->data;

    top = top->next;
    free(temp);

    printf("Pop : %d\n", pop_data);
    return pop_data;
}

```

```

int peek() {
    if (top == NULL) {
        printf("스택이 비어 있습니다.\n");
        return -1;
    }
    return top->data;
}

void print_stack() {
    Node* temp = top;
    printf("\n[현재 스택 상태 (Top -> Bottom)]\n");
    if (temp == NULL) {
        printf("비어있음\n");
    }
    while (temp != NULL) {
        printf("| %3d |\n", temp->data);
        printf("|-----|\n");
        temp = temp->next;
    }
    printf(" 바닥 \n\n");
}

int main() {

    push(1); push(2); push(3);
    print_stack();

    pop(); pop();
    print_stack();

    pop();
    pop();

    return 0;
}

```

```
Push: 1
Push: 2
Push: 3

[현재 스택 상태 (Top -> Bottom)]
|  3  |
|-----|
|  2  |
|-----|
|  1  |
|-----|
바닥
```

```
Pop : 3
Pop : 2

[현재 스택 상태 (Top -> Bottom)]
|  1  |
|-----|
바닥
```

```
Pop : 1
오류: 스택이 비어 있습니다! (Stack Underflow)
```